Poslik Caisse : mini-caisse Kotlin hors ligne d'abord

Lancer. Ouvrir le dossier dans Android Studio (JDK 17, SDK 36), ou bien `./gradlew installDebug` sur un émulateur ou une tablette. `app/google-services.json` pointe vers le projet Firebase `poslik-caisse-msahel` (Realtime Database europe-west1, règles dans `database.rules.json`, déployées avec `firebase deploy --only database`). **Tests :** `./gradlew testDebugUnitTest` (14 tests JVM + Robolectric : numérotation concurrente, redémarrage, impression, synchro interrompue). Le menu ⋮ permet de simuler une panne d'imprimante. Interface adaptée tablette (catalogue + ticket côte à côte) et téléphone (ticket repliable) ; pictogramme et couleur par catégorie à la place de photos (en production, l'image viendrait du catalogue).

Architecture. Compose + ViewModel → `Checkout` (le geste « Encaisser ») → Room (**source de vérité locale**) ; puis, en arrière-plan, `PrintQueue` (consommateur unique, interface `Printer`, `FakePrinter` fournie) et `SyncWorker` (WorkManager, contrainte réseau, backoff) → `SalesSyncer` → `FirebaseRemoteSalesStore`. Injection manuelle (`AppContainer`). Firebase n'est jamais sur le chemin critique de l'encaissement.

Schéma. *Room* : `register_state(id=0, registerCode, lastSeq)` · `sales(ticketId PK, registerCode, seq, createdAt, totalMillimes, printStatus PENDING|PRINTED|FAILED, printAttempts, printedAt, version, syncedVersion)` avec un index unique `(registerCode, seq)` · `sale_lines(ticketId, position, productId, name, unitPriceMillimes, quantity)`. Montants en millimes (entiers). *Firebase* : `/meta/registerCounter` · `/registers/{code}` · `/sales/{registerCode}/{ticketId}` = la vente, ses lignes, `printStatus`, `syncedAt`.

Encaisser = une transaction SQLite : lecture de `lastSeq`, +1, insertion de la vente (état `PENDING`) et de ses lignes, tout ou rien, avec `PRAGMA synchronous=FULL` pour qu'une vente validée survive à une coupure de courant. Le panier n'est vidé qu'après le commit (quelques ms), puis la main est rendue ; l'impression et la synchro sont asynchrones.

Unicité des numéros. Format `C03-000124` = **code de caisse + compteur local**. Le code est attribué **une seule fois par installation**, au premier lancement en ligne, par une transaction Firebase sur `/meta/registerCounter` (les règles imposent +1). Ensuite chaque caisse numérote seule, en ligne comme hors ligne : SQLite sérialise les écritures (testé avec 200 encaissements concurrents), et deux tablettes hors ligne en même temps ne peuvent pas collisionner puisque leurs préfixes diffèrent. En dernier rempart, les règles Firebase refusent d'écraser une vente existante avec un autre `seq`, une autre date ou un autre montant. *Alternatives écartées :* compteur global Firebase (les transactions ne fonctionnent pas hors ligne) ; numéro provisoire renuméroté à la synchro (le ticket est déjà imprimé) ; plages réservées à l'avance (une piste valable si une séquence globale est exigée, mais elle laisse des trous et des numéros hors ordre).

Limites : la numérotation est séquentielle **par caisse**, pas sur tout le magasin ; une caisse neuve doit être en ligne **une fois** (bouton désactivé et message tant qu'elle n'a pas de code) ; une réinstallation donne un nouveau code (jamais de réutilisation, donc jamais de doublon) ; si l'app meurt après la transaction Firebase mais avant l'écriture locale, un code est perdu (un trou dans les codes, sans conséquence).

Non-perte des tickets. L'état `PENDING` est écrit **avant** l'impression. La file passe le ticket à `PRINTED` ou `FAILED` ; un ticket `PRINTED` ne change plus jamais d'état (garde en SQL et dans les règles). Au démarrage, `recoverAtStartup()` renvoie les `PENDING` et les `FAILED`, jamais les `PRINTED`. Si l'app est tuée pendant l'impression, le ticket reste `PENDING` et repart : au pire une réimpression, jamais un ticket perdu (une imprimante sans accusé ne permet pas mieux). Un bouton « Réimprimer » est disponible sur les échecs.

Hors ligne et reconnexion. Sans réseau, tout fonctionne : numéro, vente, impression, historique (lu dans Room), avec l'indicateur « Hors ligne · N à synchro ». Chaque vente et chaque changement d'état d'impression incrémentent `version` et demandent une synchro WorkManager, persistée et exécutée au retour du réseau, même après un redémarrage. Le worker envoie les ventes `version > syncedVersion` par lots, avec `setValue` sur la clé `{registerCode}/{ticketId}` : **idempotent**, donc un renvoi (retry, crash avant l'accusé) écrase le même nœud au lieu d'en créer un autre (**pas de doublon**). `syncedVersion` n'est mis à jour qu'**après l'accusé serveur**, et seulement à la version envoyée : une vente modifiée pendant l'envoi repart (**pas de perte**). On ne s'appuie pas sur le cache hors ligne de Firebase : il n'est ni transactionnel ni interrogeable comme une vraie base.

Pour la production : Firebase Auth (les règles sont ouvertes par chemin et validées, sans authentification) ; une vraie imprimante ESC/POS ; une clôture de caisse.